

## Distribuirani računarski sistemi

Popravni kolokvijum – 26.05.2026.

Trajanje: 120 minuta • Ukupno: 40 poena

Ime i prezime: ..... Broj indeksa: .....

*Uputstvo: Pažljivo pročitajte sve zadatke prije nego što počnete sa radom. Predajte jedan Java projekat sa dva paketa: **todolist** (rješenje Zadatka 1) i **eventordering** (rješenje Zadatka 2). Nije dozvoljeno korišćenje interneta niti komunikacija sa drugim kandidatima.*

### Zadatak 1. (20 poena)

Napisati distribuiranu listu zadataka (*to-do* listu) koja omogućava da više korisnika istovremeno vodi i mijenja istu zajedničku listu. Sistem se sastoji od centralizovanog servera i proizvoljnog broja klijenata koji komuniciraju preko socket konekcija.

**Server.** Pri pokretanju startuje sa praznom listom i čuva njenu master verziju. Nije potrebno perzistirati listu na disk – dovoljno je držati sadržaj u memoriji kao listu stavki, gdje svaka stavka ima jedinstveni ID, opis i status (**otvoreno/završeno**). Pri svakom novom povezivanju klijentu dodjeljuje jedinstveni ID i šalje mu trenutni sadržaj liste. Prima zahtjeve za izmjenu od klijenata, ažurira master verziju i propagira novi sadržaj svim povezanim klijentima.

**Klijent.** Zna IP adresu i port servera. Po uspostavljanju veze čuva lokalnu kopiju liste koju ažurira na osnovu poruka servera. Klijent obezbjeđuje jednostavan interfejs (npr. komandna linija) sa sljedećim komandama:

- **add** <opis> – dodaje novu stavku na kraj liste (status **otvoreno**).
- **done** <id> – označava stavku sa zadatim ID-em kao **završenu**.
- **view** – prikazuje trenutni sadržaj lokalne liste.
- **exit** – zatvara konekciju i izlazi iz programa.

**Sinhronizacija i zaključavanje.** Server obezbjeđuje da dva klijenta ne mogu istovremeno mijenjati istu stavku. Zahtjev za izmjenu zaključane stavke stavlja se u red čekanja i izvršava čim se stavka oslobodi.

**Logovanje.** Server vodi čitljiv log svih primljenih zahtjeva i poslatih ažuriranja, tako da se jasno vidi tok rada sistema.

### Zadatak 2. (20 poena)

Napisati distribuirani sistem od **tri procesa** (*čvora*) koji koriste **vektorski časovnik** za praćenje kauzalnih veza među događajima. Procesi komuniciraju svaki sa svakim preko socket konekcija.

Kreirati klasu **EventOrdering** kojoj se pri kreiranju proslijeđuje lista IP adresa i portova sva tri procesa u sistemu, kao i sopstveni ID procesa. Klasa izlaže metode:

- **void localEvent(String opis)** – proces bilježi lokalni događaj sa zadatim opisom; ažurira sopstveni vektorski časovnik i loguje događaj.
- **void sendMessage(int targetId, String poruka)** – šalje poruku zadatom procesu uz vrijednost vektorskog časovnika i loguje događaj slanja.

Pri prijemu poruke proces ažurira svoj vektorski časovnik po pravilu vektorskog časovnika i loguje događaj prijema.

**Tok rada.** Pokrenuti simulaciju na **tri čvora**. Svaki čvor mora generisati nekoliko lokalnih događaja i razmijeniti nekoliko poruka sa ostala dva čvora.

**Logovanje.** Svaki proces vodi i **štampa čitljiv lokalni log** svih događaja (lokalni, slanje, prijem) sa pratećim vrijednostima vektorskog časovnika.

**Demonstracija konzistentnosti.** Nakon završene razmjene poruka, svaki čvor za **svaki par događaja iz sopstvenog loga** ispisuje njihov međusobni odnos:

- $a \rightarrow b$  – prvi događaj se desio prije drugog,
- $b \rightarrow a$  – drugi događaj se desio prije prvog,
- $a \parallel b$  – događaji su konkurentni.

Poređenje se vrši isključivo na osnovu vektorskih časovnika ( $a \rightarrow b$  akko  $VC(a) < VC(b)$ ). Logovi sva tri čvora moraju biti međusobno konzistentni: za isti par događaja koji se pojavljuje u logovima više čvorova, ispisani odnos mora biti isti.